Exercise 9

EM Practicals

Olesia Galynskaia 12321492

2025

Fix the random seed for reproducibility

```
set.seed(12321492)
knitr::opts_chunk$set(echo = TRUE, message = FALSE, warning = FALSE, fig.width = 6, fig.height = 4, dpi
```

Task 19

Packages

```
# install.packages(c("mvtnorm", "MASS"))
library(mvtnorm)
library(MASS)
```

1. Sampler for Gaussian Mixture Model

I implement a function that generates a random sample from a Gaussian Mixture Model (GMM).
The function first draws a component label for each observation according to the mixture weights.
Then it draws the observation from the multivariate normal distribution that corresponds to the selected component.
Finally, it returns both the generated data matrix and the vector of true component labels.

```
sample_gmm <- function(n, mus, Sigmas, weights, seed = NULL) {
  if (!is.null(seed)) set.seed(seed)

  K <- length(mus)
  stopifnot(length(Sigmas) == K)
  stopifnot(length(weights) == K)
  stopifnot(abs(sum(weights) - 1) < 1e-10)

  d <- length(mus[[1]])
  for (k in 1:K) {
    stopifnot(length(mus[[k]]) == d)
    stopifnot(all(dim(Sigmas[[k]]) == c(d, d)))
  }
```

```
  # sample cluster labels
  z <- sample.int(K, size = n, replace = TRUE, prob = weights)

  # sample points conditional on cluster
  X <- matrix(NA_real_, nrow = n, ncol = d)
  for (k in 1:K) {
    idx <- which(z == k)
    if (length(idx) > 0) {
      X[idx, ] <- rmvnorm(n = length(idx), mean = mus[[k]], sigma = Sigmas[[k]])
    }
  }

  list(X = X, z = z)
}
```

2. EM for GMM

I implement the EM algorithm to estimate the parameters of a Gaussian Mixture Model with a fixed number of components K.
In the E-step, we compute the responsibilities, which are the posterior probabilities that each observation belongs to each component.
In the M-step, we update the mixture weights, component means, and covariance matrices using these responsibilities.
I repeat E-steps and M-steps until the change in log-likelihood becomes smaller than a tolerance threshold or until the maximum number of iterations is reached.

Function to compute error measures

```
# covariance update
ridge_pd <- function(S, eps = 1e-6) {
  S <- (S + t(S)) / 2
  S + diag(eps, nrow(S))
}

gmm_loglik <- function(X, mus, Sigmas, weights) {
  n <- nrow(X); K <- length(mus)
  dens <- matrix(0, n, K)
  for (k in 1:K) {
    dens[, k] <- weights[k] * dmvnorm(X, mean = mus[[k]], sigma = Sigmas[[k]])
  }
  sum(log(rowSums(dens)))
}

em_gmm <- function(X, K,
                   init = list(weights = rep(1 / K, K), mus = NULL, Sigmas = NULL),
                   max_iter = 200, tol = 1e-6, ridge = 1e-6, verbose = TRUE) {

  X <- as.matrix(X)
  n <- nrow(X); d <- ncol(X)
```

```r
# init
weights <- init$weights
if (is.null(weights)) weights <- rep(1 / K, K)
weights <- as.numeric(weights)
weights <- weights / sum(weights)

if (is.null(init$mus)) {
  # pick K random points as initial means
  mus <- lapply(1:K, function(k) X[sample.int(n, 1), ])
} else {
  mus <- init$mus
}

if (is.null(init$Sigmas)) {
  S0 <- cov(X)
  Sigmas <- replicate(K, ridge_pd(S0, ridge), simplify = FALSE)
} else {
  Sigmas <- init$Sigmas
}

# EM loop
loglik_hist <- numeric(0)
resp <- matrix(0, n, K)

ll_old <- -Inf

for (iter in 1:max_iter) {

  # E-step: responsibilities
  dens <- matrix(0, n, K)
  for (k in 1:K) {
    dens[, k] <- weights[k] * dmvnorm(X, mean = mus[[k]], sigma = Sigmas[[k]])
  }
  denom <- rowSums(dens)
  # avoid division by zero
  denom[denom == 0] <- .Machine$double.eps
  resp <- dens / denom

  # M-step: update parameters
  Nk <- colSums(resp)  # size K
  weights <- Nk / n

  mus <- lapply(1:K, function(k) {
    colSums(resp[, k] * X) / Nk[k]
  })

  Sigmas <- lapply(1:K, function(k) {
    X_centered <- sweep(X, 2, mus[[k]], "-")
    # weighted covariance
    S <- matrix(0, d, d)
    for (i in 1:n) {
      xi <- X_centered[i, , drop = FALSE]
      S <- S + resp[i, k] * (t(xi) %*% xi)
```

```
      }
      S <- S / Nk[k]
      ridge_pd(S, ridge)
    })

    # log-likelihood + convergence
    ll_new <- gmm_loglik(X, mus, Sigmas, weights)
    loglik_hist <- c(loglik_hist, ll_new)

    if (verbose) {
      cat(sprintf("iter %3d | loglik = %.6f\n", iter, ll_new))
    }

    if (abs(ll_new - ll_old) < tol) break
    ll_old <- ll_new
  }

  list(
    weights = weights,
    mus = mus,
    Sigmas = Sigmas,
    resp = resp,
    loglik = tail(loglik_hist, 1),
    loglik_hist = loglik_hist,
    iters = length(loglik_hist)
  )
}
```

3. Generating 1000 samples

I generate a dataset of size 1000 from the specified two-component Gaussian mixture model.
I use the given means, diagonal covariance matrices, and mixture weights.
I also fix a random seed to make the simulation reproducible.

```
# Parameters from the assignment
mu1 <- c(0, 4)
mu2 <- c(-2, 0)
Sigma1 <- diag(c(3, 0.5))
Sigma2 <- diag(c(1, 2))
w <- c(0.6, 0.4)

dat <- sample_gmm(
  n = 1000,
  mus = list(mu1, mu2),
  Sigmas = list(Sigma1, Sigma2),
  weights = w,
  seed = 12321492
)

X <- dat$X
z_true <- dat$z
head(X)
```

```
##            [,1]       [,2]
## [1,] -0.1095754 -0.1300207
## [2,] -2.8532476 -0.4801707
## [3,] -1.3516984  4.0154007
## [4,]  0.3677482  4.2735283
## [5,] -2.3140734 -1.0911711
## [6,]  0.5419005  3.2169883
```

```r
table(z_true)
```

```
## z_true
##   1   2
## 611 389
```

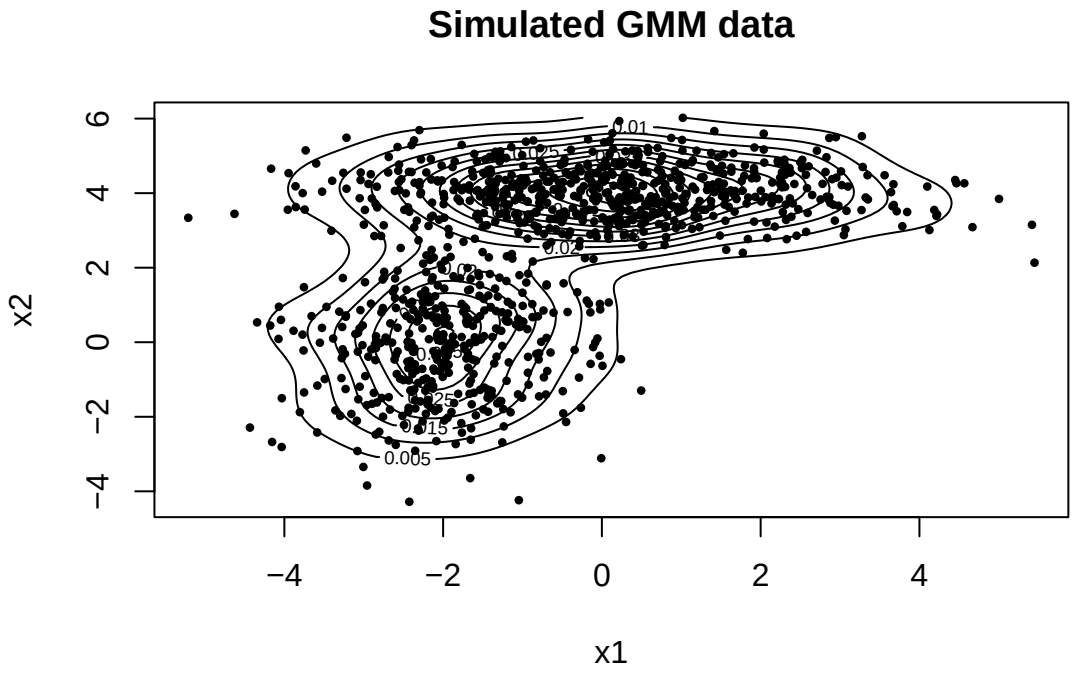
The empirical component counts are 611 observations from component 1 and 389 observations from component 2.
These proportions are close to the true mixture weights of 0.6 and 0.4, which indicates that the sampling procedure works as expected.
The first few observations show reasonable values and no obvious numerical issues.

4. Visualization

I visualize the simulated data by drawing a scatter plot and overlaying contour lines of a two-dimensional kernel density estimate.
This allows us to see the two main regions of high probability mass and check whether the sample visually matches the expected mixture structure.

```r
plot(X, pch = 16, cex = 0.6, main = "Simulated GMM data", xlab = "x1", ylab = "x2")

kd <- kde2d(X[,1], X[,2], n = 100)
contour(kd, add = TRUE)
```



The scatter plot together with the kernel density contours shows two clearly separated regions of high density. One cluster is centered around higher values of the second coordinate, while the other cluster is located lower and slightly to the left.
The contour lines follow elliptical shapes, which is consistent with the Gaussian structure of the components. Overall, the visualization confirms that the simulated data reflects the intended mixture of two Gaussian distributions.

5. EM with three different initializations

1. Random means from the data

I initialize the component means by randomly selecting observations from the dataset.
I use equal mixture weights and the empirical covariance matrix of the data for both components.

```
init1 <- list(
  weights = c(0.5, 0.5),
  mus = list(
    X[sample(1:nrow(X), 1), ],
    X[sample(1:nrow(X), 1), ]
  ),
  Sigmas = list(cov(X), cov(X))
)

fit1 <- em_gmm(X, K = 2, init = init1, verbose = FALSE)

fit1$loglik
```

```
## [1] -3661.488
```

```r
fit1$iters
```

```
## [1] 19
```

```r
fit1$weights
```

```
## [1] 0.3936201 0.6063799
```

```r
fit1$mus
```

```
## [[1]]
## [1] -1.9654871 -0.0325069
##
## [[2]]
## [1] 0.1448691 4.0443531
```

With random initialization, the EM algorithm converges to a solution with a final log-likelihood of -3661.488 in 19 iterations.
The estimated mixture weights are close to the true values, and the estimated means are also close to the true cluster centers.
This shows that even with random starting values, the EM algorithm is able to recover the correct mixture structure, although it requires a moderate number of iterations to converge.

2. Means close to the true centers

I initialize the means close to the true cluster centers used to generate the data.
This initialization represents a well-informed starting point and should lead to fast convergence and a good solution.

```r
init2 <- list(
  weights = c(0.6, 0.4),
  mus = list(
    c(0, 4),
    c(-2, 0)
  ),
  Sigmas = list(diag(c(3, 0.5)), diag(c(1, 2)))
)

fit2 <- em_gmm(X, K = 2, init = init2, verbose = FALSE)

fit2$loglik
```

```
## [1] -3661.488
```

```r
fit2$iters
```

```
## [1] 14
```

```r
fit2$weights
```

```
## [1] 0.6063792 0.3936208
```

```r
fit2$mus
```

```
## [[1]]
## [1] 0.1448709 4.0443541
##
## [[2]]
## [1] -1.96548606 -0.03250118
```

When the algorithm is initialized with means close to the true generating centers, it converges faster, requiring only 14 iterations.
The final log-likelihood is -3661.488, which is the same as for the other initializations. The estimated parameters closely match the true mixture weights and means.
This indicates that a good initialization leads to faster convergence while reaching the same optimal solution.

3. Poor initialization

I choose an intentionally poor initialization where both component means are placed close to each other. This setup tests whether the EM algorithm can still separate the clusters or whether it converges to a worse local optimum.

```r
init3 <- list(
  weights = c(0.5, 0.5),
  mus = list(
    c(0, 0),
    c(0.5, 0.5)
  ),
  Sigmas = list(diag(2), diag(2))
)

fit3 <- em_gmm(X, K = 2, init = init3, verbose = FALSE)

fit3$loglik
```

```
## [1] -3661.488
```

```r
fit3$iters
```

```
## [1] 21
```

```r
fit3$weights
```

```
## [1] 0.3936328 0.6063672
```

```
fit3$mus
```

```
## [[1]]
## [1] -1.96546872 -0.03240303
##
## [[2]]
## [1] 0.1449016 4.0443714
```

With a poor initialization where the component means are placed close to each other, the EM algorithm still converges to the same final log-likelihood of -3661.488.
However, it requires more iterations (21 iterations) compared to the other cases.
Despite the unfavorable starting point, the algorithm successfully separates the clusters and recovers reasonable parameter estimates, but convergence is slower.

Comparison table

```
comparison <- data.frame(
  Initialization = c("Random", "Near true values", "Poor"),
  LogLikelihood = c(fit1$loglik, fit2$loglik, fit3$loglik),
  Iterations = c(fit1$iters, fit2$iters, fit3$iters)
)

comparison
```

```
##     Initialization LogLikelihood Iterations
## 1           Random     -3661.488         19
## 2 Near true values     -3661.488         14
## 3             Poor     -3661.488         21
```

All three initializations lead to the same final log-likelihood value of -3661.488, which suggests that the EM algorithm converges to the same maximum of the likelihood function in all cases.
The main difference between the runs is the number of iterations required for convergence.

The best initialization is the one with means close to the true centers.
It reaches the same final solution as the other initializations but converges in the smallest number of iterations.
This makes it the most efficient choice, as it reduces computation time while achieving the same quality of fit.

6. Applying EM to the iris data set

Data preparation

I load the iris data set and use only the numerical features. The true species labels are stored separately for evaluation.

```
data(iris)

X_iris <- as.matrix(iris[, 1:4])
y_true <- iris$Species
```

EM fit with K = 3, 4, and 5

I run the EM algorithm with different numbers of clusters to study how the clustering changes when the assumed number of components is correct or incorrect.

```
set.seed(12321492)

fit3 <- em_gmm(X_iris, K = 3, verbose = FALSE)
fit4 <- em_gmm(X_iris, K = 4, verbose = FALSE)
fit5 <- em_gmm(X_iris, K = 5, verbose = FALSE)
```

Cluster assignments

I assign each observation to the cluster with the highest posterior responsibility.

```
z3 <- apply(fit3$resp, 1, which.max)
z4 <- apply(fit4$resp, 1, which.max)
z5 <- apply(fit5$resp, 1, which.max)
```

Visualization

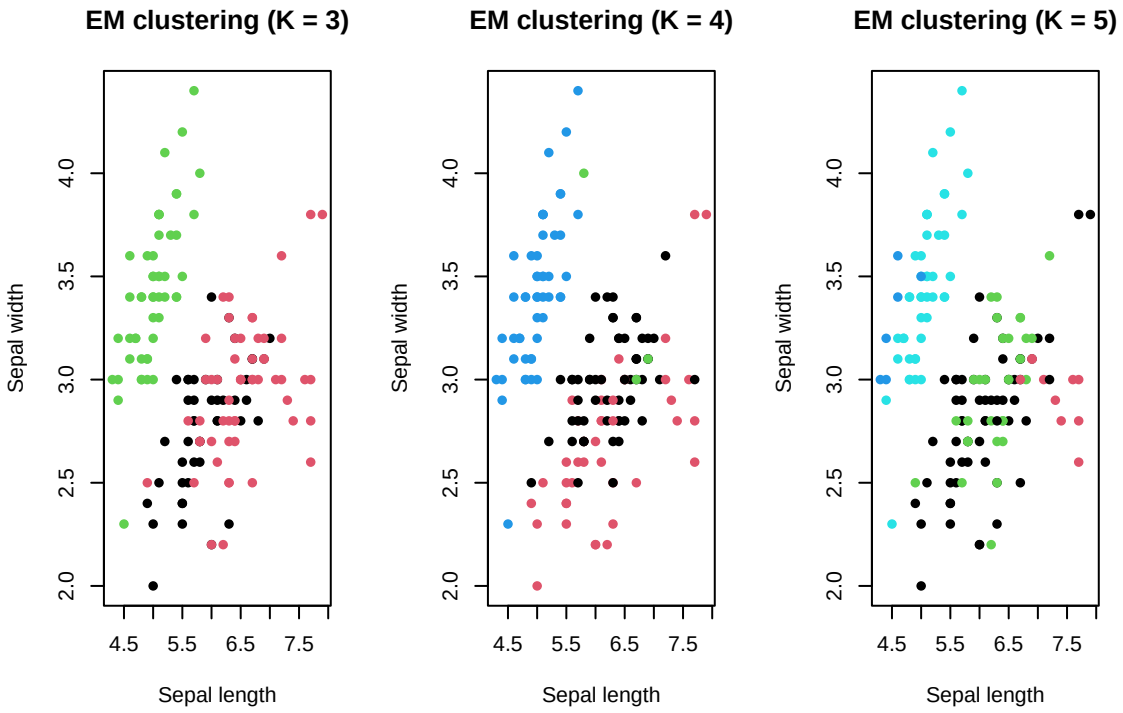
I visualize the clustering results using the first two variables for simplicity. This allows us to visually inspect how well the clusters separate the data.

```
par(mfrow = c(1, 3))

plot(X_iris[,1], X_iris[,2], col = z3, pch = 16,
     main = "EM clustering (K = 3)",
     xlab = "Sepal length", ylab = "Sepal width")

plot(X_iris[,1], X_iris[,2], col = z4, pch = 16,
     main = "EM clustering (K = 4)",
     xlab = "Sepal length", ylab = "Sepal width")

plot(X_iris[,1], X_iris[,2], col = z5, pch = 16,
     main = "EM clustering (K = 5)",
     xlab = "Sepal length", ylab = "Sepal width")
```



The plots show the EM clustering results for the iris data set using different numbers of clusters.

For K = 3, the data are grouped into three main clusters. One cluster is clearly separated from the others, while the remaining two clusters overlap, which reflects the similarity between two of the iris species.

For K = 4, the algorithm splits one of the existing groups into two smaller clusters. This leads to a more fragmented structure without a clear visual improvement in separation.

For K = 5, the data are divided into even more subclusters. The clusters become smaller and more mixed, which makes the overall structure harder to interpret.

Overall, the visualization suggests that increasing the number of clusters beyond three does not lead to clearer separation and mainly results in over-segmentation of the data.

Misclassification rate for the correct number of clusters (K = 3)

We assign each observation to the cluster with the highest posterior probability obtained from the EM algorithm with K = 3.
Then, for each cluster, we assign it to the most frequent true species label within that cluster.
The misclassification rate is computed as the proportion of observations whose assigned cluster label does not match the true species label.

```
misclassification_rate <- function(z_hat, y_true) {
  y_true <- as.factor(y_true)
  z_hat <- as.factor(z_hat)

  correct <- 0
  for (k in levels(z_hat)) {
    idx <- which(z_hat == k)
    majority_label <- names(sort(table(y_true[idx]), decreasing = TRUE))[1]
```

```
    correct <- correct + sum(y_true[idx] == majority_label)
  }
  1 - correct / length(y_true)
}

mis_rate_3 <- misclassification_rate(z3, y_true)
mis_rate_3
```

```
## [1] 0.03333333
```

For K = 3, the misclassification rate is approximately 0.033, which means that about 3.3% of the observations are incorrectly assigned.
This indicates that the EM algorithm with a Gaussian mixture model performs well on the iris data set when the correct number of clusters is used.
Most of the errors are due to overlap between the two more similar species, while the clearly separated species is classified almost perfectly.